

ANDROID DEVELOPMENT

B.E. (E&TC Engineering) | 2019 Pattern | Semester VIII | Elective V

BACKLOG EXAM — 04 December 2024 | Max Marks: 70

Q1 a) Explain basic building blocks of Android Application. [9 Marks]

Android applications are made up of four fundamental components. These are the building blocks that every app is structured around, and all of them must be declared in the AndroidManifest.xml file.

1. Activities

An Activity represents a single screen that the user sees and interacts with. Every app has at least one activity — the launcher activity — which is the first screen shown when the app opens. Activities have a well-defined lifecycle (created, started, resumed, paused, stopped, destroyed) that Android manages automatically. For example, a login screen is one activity and the home screen is another.

2. Services

A Service runs in the background without any user interface. It is used for long-running tasks such as playing music, downloading files, or syncing data. Services continue to run even when the user switches to another app. There are two types: Started Services (run independently) and Bound Services (connected to a component).

3. Broadcast Receivers

A Broadcast Receiver listens for and responds to system-wide broadcast messages. For example, when the battery is low, the screen turns off, or the network changes, Android sends a broadcast. Any app can register to receive these messages and react accordingly. Broadcast Receivers don't have a UI but can show notifications.

4. Content Providers

A Content Provider manages and shares data between applications. It provides a standardized way to access data such as contacts, calendar events, or media files. Android's own apps like Contacts and Gallery use Content Providers. Other apps can query this data using a URI (Uniform Resource Identifier).

Supporting Elements

Component	Purpose
Intents	Messaging objects used to request actions from other components
AndroidManifest.xml	Central config file that declares all components and permissions
Resources	Layouts, images, strings, and other UI assets stored in res/ folder

Q1 b) Explain Android API levels. [8 Marks]

An API Level is an integer number that uniquely identifies the version of the Android platform framework. Each new version of Android comes with new features and APIs, and the API level increases with each release.

Why API Levels Matter

API levels help developers ensure their app works correctly across different Android devices. When building an app, developers specify three key values in the AndroidManifest.xml file:

- `minSdkVersion` — The minimum API level the device must have to install the app. If a device's Android version is below this, the app won't install.
- `targetSdkVersion` — The API level the app has been tested and designed for. Android uses this to apply compatibility behavior.
- `compileSdkVersion` — The API level used to compile the code. This is set in the `build.gradle` file.

Android Version	API Level	Codename	Year
Android 4.4	19	KitKat	2013
Android 5.0	21	Lollipop	2014
Android 6.0	23	Marshmallow	2015
Android 7.0	24	Nougat	2016
Android 8.0	26	Oreo	2017
Android 9.0	28	Pie	2018
Android 10	29	Q	2019
Android 11	30	R	2020
Android 12	31	S	2021
Android 13	33	Tiramisu	2022

Google Play Store uses the API level to filter which apps are shown to which devices. A higher minimum SDK means fewer compatible devices but access to newer features. A lower minimum SDK reaches more users but restricts the features you can use.

Q2 a) Explain the structure of Android project. [9 Marks]

When a new project is created in Android Studio, it generates a standard folder structure. Understanding this structure is essential for developing Android applications.

Main Folders and Files

- `app/src/main/java/` — Contains all Java source code organized by package name. This is where Activity classes and other logic are written.
- `app/src/main/res/` — Contains all resources like layouts, images, strings, and colors. It has several sub-folders.
- `app/src/main/AndroidManifest.xml` — The central configuration file of the app. Declares all activities, services, permissions, and the app's package name.
- `app/build.gradle` — Module-level build file. Specifies the SDK versions, dependencies (libraries), and build settings.
- `build.gradle` (Project level) — Top-level build file for the entire project.

Sub-folders inside `res/`

Folder	Contents
<code>layout/</code>	XML files that define the user interface of each screen
<code>drawable/</code>	Images, icons, and XML shape files
<code>values/</code>	<code>strings.xml</code> (text), <code>colors.xml</code> (colors), <code>styles.xml</code> (themes)
<code>mipmap/</code>	App launcher icons in different sizes
<code>menu/</code>	XML files defining menu items
<code>raw/</code>	Audio, video, or other raw files

R.java (Auto-generated)

Android automatically generates a file called `R.java` that maps all resource IDs. Developers use it to reference resources in code, for example `R.layout.activity_main` for a layout file or `R.id.button1` for a button defined in XML.

Q2 b) Explain components of communication in Android. [8 Marks]

Android provides several mechanisms that allow different components (activities, services, etc.) to communicate with each other.

1. Intents

Intents are the primary way components communicate. An Intent is a message object that describes what action should be performed. There are two types:

- **Explicit Intent** — Directly names the target component. For example, opening a specific activity. Used when you know exactly which component to communicate with.
- **Implicit Intent** — Does not name a specific component. Instead, it declares a general action (like opening a URL or sharing text) and Android finds a suitable app to handle it.

2. Intent Filters

Intent filters are declared in `AndroidManifest.xml` and tell the system which intents a component can handle. For example, an activity can declare an intent filter for viewing images, so when another app wants to view an image, Android offers this app as an option.

3. Broadcast and Broadcast Receivers

A broadcast is a message sent to all registered receivers simultaneously. Android sends system broadcasts for events like low battery, network changes, or boot completion. Apps can also send custom broadcasts. A Broadcast Receiver listens for these messages and responds to them.

4. Content Providers

Content Providers allow data sharing between applications through a URI-based interface. One app can expose its data (like a database) via a Content Provider and other apps can read or write that data using ContentResolver.

5. Bound Services

An activity can bind to a service to communicate with it directly. Once bound, the activity can call methods on the service and receive results. This is useful for ongoing interactions, like controlling a music player service from an activity.

Q3 a) Write the procedure to link the activities using intents. [9 Marks]

Linking activities means navigating from one screen (Activity) to another. This is done using Intents. The procedure involves several steps:

Step 1 — Create Two Activities

In Android Studio, create two activities: MainActivity (the first screen) and SecondActivity (the destination screen). Each activity has a Java file and an XML layout file.

Step 2 — Design the Layouts

In the layout of MainActivity, add an EditText for user input and a Button to trigger navigation. In the layout of SecondActivity, add a TextView to display the received data.

Step 3 — Write Code in MainActivity

In MainActivity.java, set an OnClickListener on the button. When clicked, create an Intent object pointing to SecondActivity. Use intent.putExtra() to attach any data (like a name or message) to the intent. Finally, call startActivity(intent) to launch SecondActivity.

Step 4 — Write Code in SecondActivity

In SecondActivity.java, use getIntent() to retrieve the Intent that started it. Then use getStringExtra() or other getter methods to extract the data that was passed. Display the data in the TextView.

Step 5 — Register SecondActivity in Manifest

Every activity must be declared in AndroidManifest.xml. Add a tag for SecondActivity inside the application tag. This is usually done automatically by Android Studio when you create a new activity.

Passing Data Back to Previous Activity

If the first activity needs a result from the second activity, use startActivityForResult() instead of startActivity(). In the second activity, call setResult() with the data before calling finish(). The first activity receives the result in the onActivityResult() callback method.

Q3 b) Explain Components of screen and Adapting Display Orientation. [9 Marks]

Components of a Screen

The Android screen UI is built from the following components:

- **View** — The basic building block of any UI. A View occupies a rectangular area on the screen. Examples include TextView (for text), Button (for clicks), ImageView (for images), EditText (for input), CheckBox, RadioButton, SeekBar, and ProgressBar.
- **ViewGroup (Layouts)** — A special type of View that acts as a container holding other Views. It defines how child views are arranged. Examples include LinearLayout (arranges items in a row or column), RelativeLayout (positions items relative to each other), and ConstraintLayout (flexible positioning using constraints).
- **Window** — Every Activity draws its content into a window. The window holds the entire view hierarchy and is managed by the Android system.
- **Canvas** — Provides a surface for custom 2D drawing. Used when you need to draw shapes, lines, or custom graphics directly on screen.

Adapting to Display Orientation

Android devices support two orientations: Portrait (tall) and Landscape (wide). An app must handle orientation changes gracefully. There are several methods to do this:

- **Method 1 — Lock Orientation:** In AndroidManifest.xml, set the screenOrientation attribute of an activity to portrait or landscape. This prevents the screen from rotating but limits flexibility.
- **Method 2 — Provide Alternate Layouts:** Create a folder called res/layout-land/ and place a landscape-specific version of the layout XML file there with the same filename. Android automatically loads the correct layout based on the current orientation.

- Method 3 — Handle Config Changes in Code: By adding `configChanges='orientation|screenSize'` in the manifest, Android won't restart the activity on rotation. Instead, it calls `onConfigurationChanged()` where you can manually adjust the UI.
- Method 4 — Save and Restore State: When orientation changes, Android destroys and recreates the activity by default. Use `onSaveInstanceState()` to save data before destruction and restore it in `onCreate()` using the `savedInstanceState` bundle.

Q4 a) Explain the procedure for replacing fragments. [8 Marks]

A Fragment is a modular section of UI that can be displayed within an Activity. Replacing a fragment means swapping one fragment with another dynamically at runtime. This is done using `FragmentManager` and `FragmentTransaction`.

Step 1 — Create Fragment Classes

Create two Fragment classes, for example `FragmentA` and `FragmentB`. Each fragment has its own Java class that extends `Fragment` and overrides the `onCreateView()` method to inflate its layout XML file.

Step 2 — Create Fragment Layouts

Create separate XML layout files for each fragment, for example `fragment_a.xml` and `fragment_b.xml`. These define what the user sees when each fragment is displayed.

Step 3 — Add a Container in Activity Layout

In the Activity's XML layout file, add a `FrameLayout` with an ID (for example `fragment_container`). This acts as the placeholder where fragments will be loaded and swapped.

Step 4 — Load and Replace Fragments in Java

In the Activity's Java file, use `getSupportFragmentManager().beginTransaction()` to start a fragment transaction. Use `add()` to load the initial fragment and `replace()` to swap it with another fragment. Call `commit()` to apply the change. To allow the user to press the back button to go back to the previous fragment, call `addToBackStack()` before `commit()`.

Key Fragment Transaction Methods

Method	Purpose
<code>add()</code>	Adds a fragment to the specified container for the first time
<code>replace()</code>	Replaces the currently displayed fragment with a new one
<code>remove()</code>	Removes a fragment from the container
<code>addToBackStack()</code>	Saves the transaction so the back button can undo it
<code>commit()</code>	Finalizes and applies the transaction

Q4 b) Explain about Absolute and Relative Layouts. [10 Marks]

1. AbsoluteLayout

`AbsoluteLayout` allows you to place UI elements at exact X and Y pixel coordinates on the screen. You specify the exact position of each child view using `layout_x` and `layout_y` attributes.

This layout is considered deprecated (removed from active use) since API level 3 because it does not adapt to different screen sizes. A button placed at coordinates (100, 200) will appear at different visual positions on a small phone versus a large tablet. It is rigid and not screen-size-aware.

2. RelativeLayout

`RelativeLayout` allows you to position child views relative to each other or relative to the parent container. Instead of fixed coordinates, you describe relationships like 'this button is below the text field' or 'this label is aligned to the right of the parent'.

This makes `RelativeLayout` much more flexible and adaptable than `AbsoluteLayout`. It is widely used and works well on different screen sizes.

Common RelativeLayout Positioning Attributes

Attribute	What it does
-----------	--------------

layout_alignParentTop	Places the view at the top of the parent container
layout_alignParentBottom	Places the view at the bottom of the parent container
layout_centerInParent	Centers the view both horizontally and vertically in the parent
layout_below	Places the view directly below the specified view
layout_above	Places the view directly above the specified view
layout_toRightOf	Places the view to the right of the specified view
layout_toLeftOf	Places the view to the left of the specified view
layout_centerHorizontal	Centers the view horizontally in the parent

Comparison

Feature	AbsoluteLayout	RelativeLayout
Positioning	Fixed X, Y pixel coordinates	Relative to parent or sibling views
Screen adaptability	Poor — breaks on different screens	Good — works across screen sizes
Status	Deprecated	Actively supported and used
Flexibility	None	High

Q5 a) Explain Android Menus. [9 Marks]

Menus provide a way to present additional options and actions to the user without cluttering the main screen. Android supports three types of menus:

1. Options Menu

The Options Menu appears when the user taps the three-dot overflow icon in the top-right corner of the ActionBar (toolbar). It is used for global actions that apply to the entire screen, such as Settings, Help, or Search.

To implement it, you create a menu XML file in the `res/menu/` folder and inflate it in the activity by overriding `onCreateOptionsMenu()`. You handle menu item clicks in the `onOptionsItemSelected()` method.

2. Context Menu

The Context Menu appears when the user performs a long press (press and hold) on a specific UI element such as a list item or an image. It provides actions specific to that particular item, such as Copy, Delete, or Share.

To implement it, register the view for a context menu using `registerForContextMenu()`. Override `onCreateContextMenu()` to define the menu items and `onContextItemSelected()` to handle the user's choice.

3. Popup Menu

A Popup Menu is a modal menu that is anchored to a specific view, appearing below or above it. It is triggered by a button click and typically used for actions that are related to a particular UI element rather than the entire screen.

To implement it, create a `PopupMenu` object with the anchor view, inflate a menu XML file, set a click listener, and call `popup.show()` to display it.

Comparison of Menu Types

Menu Type	How Triggered	Best Used For
Options Menu	Tapping ActionBar overflow icon	Global screen-level actions (Settings, About)
Context Menu	Long pressing a specific view	Actions on a specific selected item
Popup Menu	Clicking a button	Contextual options anchored to a view

Q5 b) Explain 2D and 3D Graphics in Android. [8 Marks]

2D Graphics

Android provides multiple ways to draw two-dimensional graphics on the screen:

- **Canvas and Paint** — The most common approach. A `Canvas` object provides methods for drawing shapes, text, and images. A `Paint` object defines the visual properties such as color, stroke width, text size, and whether shapes are filled or outlined. You create a custom `View` class, override the `onDraw()` method, and use the `Canvas` to draw whatever you need. Common drawing methods include `drawCircle()`, `drawRect()`, `drawLine()`, `drawText()`, and `drawBitmap()`.
- **Drawable Resources** — You can use PNG or JPEG images placed in the `res/drawable/` folder. You can also create XML shape drawables that define geometric shapes (rectangles, ovals, lines) with colors and gradients, all without needing any image files.
- **SurfaceView** — Used for high-performance drawing, such as in games or camera previews. `SurfaceView` provides a dedicated drawing thread that runs separately from the UI thread, ensuring smooth animations.

3D Graphics

Android supports 3D graphics through OpenGL ES (Open Graphics Library for Embedded Systems). This is a standard API for rendering 2D and 3D graphics on mobile and embedded devices.

- **GLSurfaceView** — A special view that provides a surface for OpenGL rendering. It manages the OpenGL context and the rendering thread automatically.
- **GLSurfaceView.Renderer** — An interface you implement to define what gets drawn. It has three methods: `onSurfaceCreated()` called once when the surface is first created, `onSurfaceChanged()` called when the surface size changes, and `onDrawFrame()` called for every frame that needs to be rendered.
- **OpenGL ES versions** — Android supports OpenGL ES 1.0, 2.0, and 3.0. Higher versions offer more features like shaders and advanced lighting.

Comparison

Feature	2D Graphics	3D Graphics
API Used	Canvas, Paint, Drawable	OpenGL ES, GLSurfaceView
Complexity	Simple and straightforward	More complex, requires math knowledge
Typical Use Cases	UI elements, simple games, custom views	3D games, simulations, visualizations
Performance	Moderate	High — GPU accelerated

Q6 a) Explain Text View in detail. [9 Marks]

TextView is one of the most fundamental and commonly used UI widgets in Android. Its primary purpose is to display text on the screen. It is the base class for several other input widgets like EditText and Button.

Basic Usage

A TextView is defined in an XML layout file. You can set its text, size, color, style, alignment, and many other properties directly in XML using attributes.

Important XML Attributes

Attribute	Purpose
android:text	Sets the text content to display
android:textSize	Sets the font size (always use sp unit for text)
android:textColor	Sets the color of the text
android:textStyle	Sets text formatting — bold, italic, or normal
android:gravity	Controls alignment of text within the view (center, left, right)
android:maxLines	Limits the maximum number of lines shown
android:ellipsize	Determines how text is truncated when too long (end, start, middle)
android:autoLink	Automatically makes URLs, phone numbers, and emails clickable
android:fontFamily	Sets the font typeface (e.g. sans-serif, monospace)
android:lineSpacingExtra	Adds extra space between lines
android:background	Sets a background color or drawable behind the text
android:padding	Adds space between the text and the border of the view

Spannable Text (Rich Text)

TextView supports Spannable strings which allow different parts of the same text to have different styles. For example, you can make one word red, another word bold, and another word a clickable link, all in the same TextView. This is done using SpannableString along with span classes like ForegroundColorSpan, StyleSpan, or URLSpan.

AutoCompleteTextView

AutoCompleteTextView is a subclass of TextView that shows a dropdown list of suggestions as the user types. You provide it with an ArrayAdapter containing the suggestions. It is commonly used in search bars and form fields.

Q6 b) Explain i) Grid View ii) Image Switcher View. [8 Marks]

i) GridView

GridView is a ViewGroup widget that displays items arranged in a two-dimensional, scrollable grid — like rows and columns. It is commonly used to display galleries of images, app icons, or product thumbnails.

GridView works with an Adapter that provides the data and the individual item views. You specify the number of columns using the numColumns attribute. You can also control horizontal and vertical spacing between items, and the column width. When the user taps an item, the OnItemClickListener is triggered giving you the position of the selected item.

For displaying images in a grid, a custom adapter (usually extending `BaseAdapter`) is created. The adapter's `getView()` method is called for each grid cell and returns an `ImageView` configured with the correct image.

ii) ImageSwitcher View

`ImageSwitcher` is a widget that displays images and provides smooth transition animations when switching from one image to another. It extends the `ViewSwitcher` class and is ideal for building image galleries or slideshows.

To use an `ImageSwitcher`, you implement the `ViewFactory` interface and define a `makeView()` method that returns an `ImageView`. This tells the `ImageSwitcher` how to create the view that will show each image. You also set in-animation and out-animation to control how images transition (such as fade in/fade out or slide in/slide out). When you call `setImageResource()` with a new image, the transition animation plays automatically.

A typical use case is a gallery where Previous and Next buttons let the user browse through a list of images with smooth animated transitions between them.

Q7 a) Write the steps for inserting data and updating data into the database. [9 Marks]

Android has SQLite built in as its local database system. To work with a database, the recommended approach is to use the SQLiteOpenHelper class which manages database creation and version upgrades.

Step 1 — Create a DatabaseHelper Class

Create a class that extends SQLiteOpenHelper. In the constructor, provide the database name and version number. Override two methods: onCreate() which runs when the database is first created and should contain the SQL statement to create your tables, and onUpgrade() which runs when you increase the database version and should handle schema changes.

Step 2 — Insert Data

To insert a record, get a writable instance of the database using getWritableDatabase(). Create a ContentValues object and use its put() method to add each column name and value as a key-value pair. Call db.insert() passing the table name and the ContentValues object. The method returns the row ID of the newly inserted record, or -1 if the insert failed. Always close the database after the operation.

Step 3 — Update Data

To update an existing record, again get a writable database and create a ContentValues object with the new values. Call db.update() passing the table name, the ContentValues, a WHERE clause (to identify which records to update), and the WHERE arguments. The method returns the number of rows that were updated. This lets you confirm whether the update succeeded.

Step 4 — Query / Read Data

To read data, get a readable database using getReadableDatabase(). Call db.rawQuery() with a SELECT SQL statement. This returns a Cursor object. Move through the results using cursor.moveToFirst() and cursor.moveToNext(). Use cursor.getString(), cursor.getInt(), etc. to read each column's value. Always close the Cursor after you're done.

Step 5 — Delete Data

To delete records, call db.delete() with the table name and a WHERE clause specifying which records to remove. To delete all records, pass null as the WHERE clause. The method returns the number of rows deleted.

Q7 b) Explain about saving data to internal and external storage. [9 Marks]

Android provides two types of file storage for saving data that doesn't fit neatly into a database:

1. Internal Storage

Internal storage is private to the application. Files saved here cannot be accessed by other apps or the user (without root access). The files are automatically deleted when the app is uninstalled. No special permissions are required. The storage location is in the device's internal memory at /data/data/packageName/files/.

To write a file to internal storage, use openFileOutput() which returns a FileOutputStream. Write your data as bytes using the stream. To read a file, use openFileInput() which returns a FileInputStream. Wrap it in a BufferedReader to read line by line. Files can be stored in MODE_PRIVATE (only this app) or MODE_APPEND (add to existing file).

2. External Storage

External storage is shared storage that can be accessed by other apps and the user directly. It may be on an SD card or the device's external storage partition. Files here persist even after app uninstallation unless saved to the app-specific external directory. For older Android versions (below Android 10), you need READ_EXTERNAL_STORAGE and WRITE_EXTERNAL_STORAGE permissions in the manifest.

Before writing to external storage, always check if it's available by calling `Environment.getExternalStorageState()` and verifying it equals `MEDIA_MOUNTED`. Then use `getExternalFilesDir()` to get the app-specific external directory. Use standard Java File I/O (`FileWriter`, `FileReader`, `BufferedReader`) to write and read files.

Comparison

Feature	Internal Storage	External Storage
Privacy	Private — only accessible by your app	Shared — accessible by other apps and users
Permissions	None required	Required for Android versions below 10
Availability	Always available	May not be available if SD card removed
On uninstall	Deleted automatically	Only app-specific folder is deleted
Typical size	Limited internal memory	Larger space on SD card

Q8 a) Write short note on Database Operations. [9 Marks]

Android uses SQLite as its embedded relational database. SQLite is lightweight, serverless, and stores the entire database as a single file on the device. The `android.database.sqlite` package provides all the tools needed to perform database operations.

Core Database Operations (CRUD)

- **CREATE** — Creating the database and tables is done using `execSQL()` with a `CREATE TABLE` SQL statement. The `SQLiteOpenHelper` class handles this automatically in its `onCreate()` method the first time the app runs.
- **INSERT** — Adding new records is done using `ContentValues` (a key-value map of column names to values) and the `db.insert()` method. Alternatively, you can use `db.execSQL()` with a raw `INSERT INTO` SQL string.
- **READ/QUERY** — Retrieving data is done using `db.rawQuery()` with a `SELECT` statement, which returns a `Cursor`. The `Cursor` acts like a pointer that moves through the result rows. You call `moveToFirst()` to go to the first row, then `moveToNext()` to iterate. Methods like `getString()`, `getInt()`, and `getDouble()` extract values from each column.
- **UPDATE** — Modifying existing records uses `ContentValues` with new values and `db.update()` with a `WHERE` condition to identify which rows to change.
- **DELETE** — Removing records uses `db.delete()` with a `WHERE` clause. To delete all rows, pass null for the `WHERE` condition.

SQLiteOpenHelper (Best Practice)

Rather than managing the database directly, it is best practice to create a helper class extending `SQLiteOpenHelper`. This class handles database creation in `onCreate()` and schema changes in `onUpgrade()`. When you increase the version number, `onUpgrade()` is called automatically, allowing you to safely alter or recreate tables without breaking existing installations.

Important Points

- Always close `Cursor` and `SQLiteDatabase` objects after use to avoid memory leaks.
- Database operations that may take time should be done in a background thread to avoid blocking the UI.
- Use parameterized queries (with `?` placeholders) instead of string concatenation to prevent SQL injection attacks.

Q8 b) Write Short Note on: i) Google Maps ii) Monitoring a Location. [9 Marks]

i) Google Maps in Android

Google Maps API allows developers to embed interactive Google Maps directly into Android applications. Users can view maps, add markers at locations, draw routes, and interact with the map using touch gestures like pinch-to-zoom and swipe-to-pan.

To use Google Maps, you must first obtain an API Key from the Google Cloud Console. This key is added to the `AndroidManifest.xml` file. The Google Play Services Maps library must be added as a dependency in `build.gradle`.

In the layout, you add a `SupportMapFragment` which provides the map view. In the activity, you implement the `OnMapReadyCallback` interface. When the map is ready, you receive a `GoogleMap` object that lets you add markers using `MarkerOptions`, move the camera to a location using `CameraUpdateFactory`, and set the map type (Normal, Satellite, Hybrid, or Terrain).

ii) Monitoring a Location

Location monitoring means continuously tracking the device's geographic position using GPS or network-based location. Android provides the `LocationManager` class for this purpose.

To use location services, add `ACCESS_FINE_LOCATION` (for GPS precision) and `ACCESS_COARSE_LOCATION` (for network-based position) permissions in the manifest. At runtime on Android 6.0 and above, you must also request these permissions from the user.

You create a `LocationListener` that has callback methods: `onLocationChanged()` is called whenever the device's position changes — this is where you receive the new `Location` object with latitude and longitude. `onProviderEnabled()` and `onProviderDisabled()` are called when the user enables or disables location services.

You register the listener with `LocationManager.requestLocationUpdates()`, specifying the provider (`GPS_PROVIDER` or `NETWORK_PROVIDER`), the minimum time interval between updates in milliseconds, and the minimum distance change in meters that triggers an update. To stop monitoring and save battery, call `removeUpdates()` when the activity is paused or stopped.

Provider	Source	Accuracy	Battery Impact
GPS_PROVIDER	GPS satellites	High (within meters)	High
NETWORK_PROVIDER	Wi-Fi and cell towers	Medium (within 100m)	Low
PASSIVE_PROVIDER	Uses other apps' updates	Varies	Very Low